Prompt:
Role
You are an expert Academic Editor and Senior Software Architect specializing in Systems Programming (Java and Rust) and Distributed Systems. Your goal is to refine a Master's Thesis draft into a professional Computer Science paper.
Task
Refine the provided text to improve its academic tone, technical precision, and logical flow. The thesis focuses on the modernization and reimplementation of a Virtual Resource Management (VRM) system from Java to Rust, including architectural extensions.
Constraints & Style Guidelines
1. **Tone:** Maintain a formal, objective, and analytical academic style. Use the passive voice where appropriate for technical descriptions, but keep active voice for system "actions."
2. **Terminology:** Use standard systems programming and scheduling terminology (e.g., "abstraction layer," "state synchronization," "deterministic scheduling," "resource orchestration").
3. **Clarity:** Ensure the relationship between components (e.g., Master ADC, Acl, and RMS Connectors) is logically sound.
4. **Correction:** Fix all grammatical errors, specifically focusing on:
- Distinguishing between "then" (time) and "than" (comparison).
- Subject-verb agreement.
- Correct use of technical pluralization (e.g., "criteria" vs "criterion").
5. **Rust Context:** Since the project moved to Rust, subtly highlight concepts like safety, concurrency, or performance where the architecture implies it.
6. **Scope:** Do not add unnecessary text. Be professional but concise. Also do not use - or ; if it is possible. Additionaly, do not replace acronyms.
7. **Return Type:** Latex format
Do not add new information.
Draft Input
\section{VRM-Rust Overview} \label{sec:VRMRustOverview}
The system architecture allows for atomic task or workflow submission from \rstruct{Clients}, which are registered within the system by unique identifiers. Submitted resource requests are preprocessed into a structured format to enable efficient scheduling. The \rstruct{VrmManager} orchestrates this process and submits unprocessed workflows or atomic tasks to the \emph{Master} \rstruct{ADC}, which serves as the entry point of the system. The \emph{Master} \rstruct{ADC} then processes the request, distinguishing between a workflow and an atomic task. Atomic tasks are forwarded directly to the \rstruct{VrmComponentManager} of the

\emph{Master} \rstruct{ADC}, Workflows are instead forwarded to the \rtrait{WorkflowScheduler}, which performs a feasibility analysis to determine whether all tasks within the workflow can be handled by the system before the entire workflow is submitted via the \rstruct{VrmComponentManager}. \\

Output:

\section{VRM-Rust Overview} \label{sec:VRMRustOverview}

The system architecture facilitates the submission of atomic tasks or workflows from \rstruct{Clients}, which are registered within the system via unique identifiers. Upon submission, resource requests are preprocessed into a structured format to ensure efficient scheduling and memory safety. The \rstruct{VrmManager} orchestrates this process and transmits unprocessed workflows or atomic tasks to the \emph{Master} \rstruct{ADC}, which serves as the primary system entry point. The \emph{Master} \rstruct{ADC} then evaluates the request to distinguish between a workflow and an atomic task. The system forwards atomic tasks directly to the \rstruct{VrmComponentManager} of the \emph{Master} \rstruct{ADC}. Conversely, workflows are directed to the \rtrait{WorkflowScheduler} for a feasibility analysis. This process determines whether the system can accommodate all tasks within the workflow before the entire request is submitted via the \rstruct{VrmComponentManager}.